Smart Pointers and Ownership

Who owns an object: `unique_ptr`, `shared_ptr` and `weak_ptr`, the control block, cycles, and how to pass ownership around. Builds on RAII and Move Semantics.

Who owns it? The three smart pointers

RAII wrappers around `new` / `delete` (`<memory>`) that make **ownership explicit in the type**. The owner is whoever must free the object.

TYPE	OWNERSHIP	COPYABLE?	COST
`std::unique_ptr<T>`	exclusive: one owner	**no**, only movable	same as a raw pointer
`std::shared_ptr<T>`	shared: reference counted, freed when the last owner goes	yes (bumps the count)	extra control block, atomic count
`std::weak_ptr<T>`	none: observes a `shared_ptr` without keeping it alive	yes	small
raw pointer / reference	none: "I use it, I do not own it"	yes	none

> **DEFAULT** A plain value member or local first. Then `unique_ptr` when it must live on the heap (polymorphism, large, optional). `shared_ptr` only for truly shared ownership.

A FUNCTION THAT...	TAKES
only uses the object	`const Widget&` (or `Widget*` if it may be null): never the smart pointer
takes over ownership	`std::unique_ptr<Widget>` by value: the caller must `std::move`
keeps a share	`std::shared_ptr<Widget>` by value
gives ownership to the caller	returns `std::unique_ptr<Widget>` (a factory)

```
std::unique_ptr<Widget> makeWidget(std::string n);        // factory: sole ownership to the caller
void use(const Widget& w);                                // just uses: pass the object, not the owner
void take(std::unique_ptr<Widget> w);                     // takes ownership

auto w = makeWidget("alpha");
use(*w);
take(std::move(w));                                       // w is now null
```

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 12.1 Dynamic memory and smart pointers (PDF pages 479 to 504)

Programming: Principles and Practice (Stroustrup)

- 18.4 to 18.5 Resources and exceptions; resource-management pointers (RAII, unique_ptr) (PDF pages 542 to 553)

C++ Move Semantics (Josuttis)

- Chapter 13 Move-only types (PDF pages 213 to 218)

`unique_ptr` in detail

```
auto p = std::make_unique<Widget>(1, "a");    // preferred over `new`
// auto q = p;                                // ERROR: unique_ptr cannot be
copied
auto q = std::move(p);                        // OK: ownership moves, p is now
null
```

- **Zero overhead:** with the default deleter `sizeof(unique_ptr<T>) == sizeof(T*)` (checked). A function-pointer deleter makes it twice as big; a stateless lambda deleter adds nothing.
- **Custom deleter** for anything that is not `delete`: `std::unique_ptr<std::FILE, decltype(closer)> f(std::fopen( ... ), closer);` closes the file on scope exit.
- **Arrays:** `std::make_unique<int[]>(3)` uses `delete[]`. Prefer `std::vector` unless you need a fixed block.
- **Polymorphism:** `unique_ptr<Base>` holding a `Derived` needs a **virtual destructor** on `Base`, as with any `delete` through a base pointer.
- `.get()` returns the raw pointer without giving up ownership; `.release()` gives it up (you must delete it); `.reset()` destroys the object now.

KEY POINT Why `make_unique`? No naked `new`, the type is written once, and before C++17 `f(unique_ptr<A>(new A), g())` could leak when `g()` threw between the two steps.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 12.1 Dynamic memory and smart pointers (PDF pages 479 to 504)

Effective Modern C++ (Meyers)

- Item 18, Use std::unique_ptr for exclusive-ownership resource management (PDF pages 136 to 143)
- Item 21, Prefer make_unique and make_shared to new (PDF pages 157 to 165)

`shared_ptr` and the control block



Object freed when strong count hits 0. weak_ptr.lock() gives a shared_ptr or null.

unique_ptr has no control block: zero overhead, one owner.

```
auto s1 = std::make_shared<Widget>(2, "b");  // one allocation for object + control block
auto s2 = s1;                                // two owners, use_count() == 2
```

- A `shared_ptr` is **two pointers**: to the object and to a **control block** holding the strong count, the weak count and the deleter.
- `make_shared` allocates object and control block together (one allocation). `shared_ptr<T>(new T)` allocates twice.
- The deleter is not part of the `shared_ptr` type (unlike `unique_ptr`): type-erased in the control block.
- The counts are **atomic**: copying or destroying `shared_ptr`s on several threads is safe. The **object** is not protected, and changing the same `shared_ptr` variable from two threads is a race.
- Copying costs an atomic increment. On hot paths pass `const shared_ptr&` or the object itself, not a copy per call.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 12.1 Dynamic memory and smart pointers (PDF pages 479 to 504)

Effective Modern C++ (Meyers)

- Item 19, Use std::shared_ptr for shared-ownership resource management (PDF pages 143 to 152)

Reference cycles and `weak_ptr`

If `A` holds a `shared_ptr<B>` and `B` holds a `shared_ptr<A>`, each keeps the other alive: **neither is ever destroyed**. Compiled: after the scope, both objects were still alive (leaked). Make the back link (child to parent) a `weak_ptr`: both were destroyed.

```
struct Node { std::shared_ptr<Owner> parent; };        // cycle: leak
struct WNode { std::weak_ptr<OwnerW> parent; };        // weak back-pointer: freed

std::weak_ptr<Widget> w = s1;                          // observe only: does not change use_count
if (auto sp = w.lock()) { /* still alive: use sp for as long as you hold it */ }   // lock() gives a shared_ptr or null
w.expired();                                           // true once every owner is gone
```

- `lock()` is the only way to use the object: it gives a `shared_ptr` that keeps it alive while you work, or null.
- Typical uses: parent links, caches, observer lists that must not keep subscribers alive.
- A `weak_ptr` keeps the **control block** alive (not the object) until the last weak reference goes.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 12.1 Dynamic memory and smart pointers (PDF pages 479 to 504)

Effective Modern C++ (Meyers)

- Item 20, Use std::weak_ptr for shared_ptr-like pointers that can dangle (PDF pages 152 to 157)

Pitfalls and `enable_shared_from_this`

ONE CONTROL BLOCK PER OBJECT

```
auto raw = new Widget;
std::shared_ptr<Widget> a(raw);
std::shared_ptr<Widget> b(raw);        // a second, independent control
block: double delete!
```

Never build a second `shared_ptr` from a raw pointer or from `this`. Copy an existing one.

When a member function needs a `shared_ptr` to its own object (to register itself, to hand to a callback), inherit `std::enable_shared_from_this` and call `shared_from_this()`:

```
struct Session : std::enable_shared_from_this<Session> {
    std::shared_ptr<Session> self() { return shared_from_this(); }   // a
safe second owner, SAME control block
};
auto s = std::make_shared<Session>();
auto t = s->self();                    // use_count() == 2
Session raw; raw.self();               // throws std::bad_weak_ptr: no
shared_ptr owns it (compiled)
```

PITFALL	FIX
`shared_ptr` cycle	`weak_ptr` for the back link
`shared_ptr` from `this` or a raw pointer	`enable_shared_from_this`
`shared_ptr` by default "to be safe"	`unique_ptr`: you can always convert to shared later
storing `.get()` somewhere that outlives the owner	keep the smart pointer, or a `weak_ptr`
`delete` on a `.get()` result	never: the smart pointer will delete it again